

LeetCode 1343

Number of Sub-arrays of Size K and Average Greater than or Equal to Threshold

[LinkedIn](#)[Turo](#)

Problem Breakdown & Core Logic

The goal is to find the number of sub-arrays of a fixed size k where the average of the elements is greater than or equal to a given *threshold*.

Instead of dividing the sum by k for every window (which involves floating-point operations), we use a mathematical transformation for a fixed window of size k :

$$(sum / k) \geq threshold \implies sum \geq k \times threshold$$

By computing $sum_{thresh} = threshold * k$ once, we only need to compare the sum of each window against this integer value.

Algorithmic Approaches

Approach 1: Sliding Window (Optimal)

Maintain a running sum of a window of size k . Simply update the sum as the window slides across the array (add the new element, subtract the element leaving the window), and count the valid windows.

TC: $O(n)$

SC: $O(1)$

Approach 2: Prefix Sum

Compute the prefix sum array. The sum of any window can be found using:

$$\text{windowSum} = \text{prefix}[r+1] - \text{prefix}[l]$$

where $r = l + \text{windowSize} - 1$.

TC: $O(n)$

SC: $O(n)$

C++ Implementation (Sliding Window)

```
class Solution {
public:
    int numOfSubarrays(vector<int>& arr, int k, int threshold) {
        int ans = 0;
        int n = arr.size();

        int sumthresh = threshold * k; // ye krenge to bas sum check kr lenge , division
        int sum = 0;

        for(int i = 0; i < k; i++){
            sum += arr[i];
        }

        if(sum >= sumthresh) ans++;

        for(int j = k; j < n; j++){
            sum += arr[j];
            sum -= arr[j-k];
            if(sum >= sumthresh) ans++;
        }

        return ans;
    }
};
```